

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN LẬP TRÌNH TÍNH TOÁN

XÂY DỰNG ỨNG DỤNG THỰC HIỆN GIAO DỊCH TRÊN MÁY ATM

NGƯỜI HƯỚNG DẪN: GVC. ThS. TRẦN HÒ THUYẾT TIÊN

Sinh viên thực hiện:

PHAN NGUYỄN ANH VŨ

LỚP: 24T_DT2

VÕ THỊ KIM TIỀN

LỚP: 24T_DT2

NHÓM: 24.11B

Đà Nẵng, 05/2025

MỤC LỤC

MỤC LỤC.....	1
DANH MỤC HÌNH VẼ.....	3
MỞ ĐẦU.....	4
1. TỔNG QUAN ĐỀ TÀI.....	5
2. CƠ SỞ LÝ THUYẾT.....	5
2.1. Ý tưởng.....	5
2.2. Cơ sở lý thuyết.....	5
3. TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN.....	6
3.1. Phát biểu bài toán.....	6
3.2. Cấu trúc dữ liệu.....	7
3.3. Thuật toán.....	8
3.3.1. Thuật toán đọc thông tin khách hàng từ file.....	8
3.3.2. Thuật toán tìm tài khoản theo số tài khoản.....	8
3.3.3. Thuật toán lưu giao dịch vào một file.....	9
3.3.4. Thuật toán xác thực tài khoản.....	9
3.3.5. Thuật toán kiểm tra số dư.....	10
3.3.6. Thuật toán thực hiện rút tiền.....	10
3.3.7. Thuật toán thực hiện gửi tiền.....	11
4. CHƯƠNG TRÌNH VÀ KẾT QUẢ.....	12
4.1. Tổ chức chương trình.....	12
4.1.1. Sơ đồ tổ chức chương trình.....	12
4.1.2. Xây dựng chương trình.....	12
4.2. Ngôn ngữ cài đặt.....	13
4.3. Kết quả.....	14
4.3.1. Giao diện chính của chương trình.....	14

4.3.2. Kết quả thực thi của chương trình.....	15
4.3.3. Nhận xét đánh giá.....	19
5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	20
5.1. Kết luận.....	20
TÀI LIỆU THAM KHẢO.....	21
PHỤ LỤC.....	22

DANH MỤC HÌNH VẼ

Hình 1. Một số tài khoản từ file input.....	7
Hình 2. Struct CUSTOMER.....	7
Hình 3. Mạng cấu trúc.....	8
Hình 4. Sơ đồ tổ chức chương trình.....	12
Hình 5. Nguyên mẫu hàm của chương trình.....	13
Hình 6. Giao diện giới thiệu đề tài và thành viên.....	14
Hình 7. Giao diện chính của chương trình.....	14
Hình 8. Giao diện chức năng đăng nhập và xác thực tài khoản.....	15
Hình 9. Kết quả chức năng kiểm tra số dư.....	15
Hình 10. Kết quả chức năng rút tiền.....	16
Hình 11. Kết quả khi số tiền cần rút nhiều hơn số dư.....	16
Hình 12. Kết quả khi nhập số tiền cần rút không dương.....	17
Hình 13. Kết quả chức năng gửi tiền.....	17
Hình 14. Kết quả khi nhập số tiền cần gửi không dương.....	18
Hình 15. Kết quả thực hiện quá số lần giao dịch.....	18
Hình 16. Số giao dịch được lưu vào file.....	19

MỞ ĐẦU

Công nghệ thông tin trong gia đoạn hiện nay đang có những bước phát triển đột phá trên mọi lĩnh vực ở khắp nơi trên thế giới, điều xảy ra trong vòng vài năm qua ở Việt Nam là sự đầu tư mạnh mẽ vào công nghệ tiên tiến. Trong bối cảnh công nghệ ngày càng phát triển, các hệ thống tự động thực hiện giao dịch như máy ATM đóng vai trò ngày càng quan trọng trong lĩnh vực tài chính và ngân hàng.

ATM không chỉ giúp khách hàng thực hiện các giao dịch một cách nhanh chóng và tiện lợi mà còn giảm bớt áp lực cho các nhân viên văn phòng trong việc xử lý các giao dịch đơn giản hằng ngày. Với mục tiêu học tập và nghiên cứu, đề tài “ Xây dựng ứng dụng thực hiện giao dịch trên máy ATM” là nền móng quan trọng mà chúng em cần phải tìm hiểu sâu hơn, đặc biệt điều này lại càng quan trọng đối với sinh viên ngành Công nghệ thông tin chúng ta.

Mục đích đề tài là xây dựng lại một ứng dụng phần mềm mô phỏng lại các chức năng cơ bản của máy ATM như đăng nhập tài khoản, kiểm tra số dư, rút tiền, chuyển tiền,...Đi sâu vào đề tài, nhóm em dùng phương pháp nghiên cứu bao gồm việc tìm hiểu lý thuyết qua tài liệu chuyên ngành dựa trên nhu cầu của người dùng và được thiết kế dưới dạng menu. Về mục tiêu cụ thể, đề tài hướng đến việc thiết kế một ứng dụng ATM có giao dịch thân thiện, dễ sử dụng; xây dựng các chức năng giao dịch đúng chuẩn, đảm bảo an toàn và chính xác; quản lý dữ liệu tài khoản hiệu quả. Phạm vi nghiên cứu của đề tài tập trung vào phần mềm mô phỏng các thao tác giao dịch cơ bản trên máy ATM. Đối tượng nghiên cứu là quá trình giao dịch của người dùng trên hệ thống ATM bao gồm các thao tác thường gặp như rút tiền, chuyển tiền,...Chương trình được cài đặt bằng ngôn ngữ lập trình C và dựa trên kiểu cấu trúc Struct. Các chức năng của menu ở mức độ giải quyết cơ bản.

Nhóm em mong rằng đây sẽ là cơ sở để phát triển và xây dựng các chương trình phức tạp, hoàn thiện hơn để giải quyết các thao tác giao dịch một cách chính xác ở độ phức tạp cao hơn. Và thông qua đề tài này, người thực hiện có thể tiếp cận và vận dụng kiến thức vào lập trình, phân tích thiết kế hệ thống vào bài toán thực tiễn, đồng thời nâng cao kỹ năng giải quyết vấn đề và tư duy.

1. TỔNG QUAN ĐỀ TÀI

“Xây dựng ứng dụng thực hiện giao dịch trên máy ATM” nhằm mô phỏng các quy trình giao dịch cơ bản của máy ATM như: đăng nhập, kiểm tra số dư, rút tiền, chuyển khoản,...

Chương trình ứng dụng có giao diện đơn giản, tổ chức theo dạng menu lựa chọn giúp người dùng dễ sử dụng. Dữ liệu các tài khoản và lịch sử giao dịch được lưu trữ vào các file dữ liệu.

2. CƠ SỞ LÝ THUYẾT

2.1. Ý tưởng

- Mô phỏng các chức năng cơ bản của một máy ATM thực tế như: đăng nhập, kiểm tra số dư, rút tiền, chuyển tiền.
- Sử dụng cấu trúc dữ liệu struct để quản lý thông tin tài khoản, thao tác đọc/ghi file để lưu trữ và cập nhật dữ liệu giao dịch và sử dụng các thư viện chuẩn trong ngôn ngữ lập trình C.
- Các tính năng được xây dựng như sau: Nhập số tài khoản và mật khẩu của tài khoản, kiểm tra thông tin đăng nhập, lựa chọn loại giao dịch, xác nhận và cập nhật lại dữ liệu. Mỗi bước đều được thiết kế để mô phỏng quy trình thực tế nhằm tăng tính trực quan với ứng dụng ATM thực tế.
- Sử dụng menu để chọn chức năng chương trình cần thực hiện.

2.2. Cơ sở lý thuyết

1. Mảng và cấu trúc dữ liệu:
 - Mảng: được sử dụng để lưu trữ các thông tin liên quan như số tài khoản, mã PIN, và số dư tài khoản
 - Struct: được dùng để lưu trữ thông tin của từng tài khoản.
2. Thao tác với file:
 - Dữ liệu về tài khoản khách hàng được lưu trữ và truy xuất từ file văn bản (.txt hoặc .dat). Các thao tác đọc file sẽ được sử dụng để khởi tạo danh sách tài khoản khi máy ATM khởi động.
 - Các thao tác ghi file sẽ được sử dụng để cập nhật số dư hoặc lưu danh sách các giao dịch vào cuối ngày.
3. Cấu trúc điều khiển:
 - Cấu trúc rẽ nhánh (if/else, switch), cấu trúc lặp(for, while, do...while...), cấu trúc nhảy (break, continue, return) sẽ đóng vai trò quan trọng để kiểm

soát chương trình: kiểm tra thông tin đăng nhập, xử lý các yêu cầu giao dịch,...

- Chương trình giới hạn số lần đăng nhập và số giao dịch trong ngày để đảm bảo an toàn.

4. Các hàm thư viện

- a. `#include <stdio.h>`: là thư viện chuẩn về nhập/xuất dữ liệu (Standard Input/Output).
 - `printf()`: in dữ liệu ra màn hình.
 - `scanf()`: nhập dữ liệu từ bàn phím.
 - `fopen()`: Mở file.
 - `fclose()`: Đóng file.
 - `fprintf()`: ghi dữ liệu ra file.
 - `fscanf()`: đọc dữ liệu từ file.
- b. `#include <conio.h>`: là thư viện mở rộng, được dùng để xử lý các thao tác nhập xuất từ bàn phím và màn hình console.
 - `getch()`: Đọc một ký tự từ bàn phím, không hiển thị ra màn hình, không cần nhấn Enter.
- c. `#include <stdlib.h>`: là thư viện chuẩn cung cấp các hàm tiện ích cho quản lý bộ nhớ, xử lý số, chuyển đổi kiểu dữ liệu, tạo số ngẫu nhiên và điều khiển chương trình.
 - `exit()`: thoát chương trình với mã trả về.
- d. `#include <string>`: là thư viện chuẩn để xử lý chuỗi ký tự.
 - `strcmp(s1, s2)`: so sánh 2 chuỗi ký tự(trả về 0 nếu giống nhau).
- e. `#include <windows.h>` :
 - `sleep()`: tạm dừng chương trình.

3. TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

3.1. Phát biểu bài toán

Input:

- Thông tin tài khoản người dùng, được lưu trữ trong file `accounts.txt`: mã PIN, số tài khoản, tên tài khoản... Dữ liệu do người dùng nhập vào như: lựa chọn các chức năng, lựa chọn xác nhận, số tiền cần gửi, cần rút,...
- File input gồm danh sách các tài khoản khách hàng. Mỗi dòng chứa các trường thông tin cơ bản của một khách hàng.

File	Edit	View
10022006	369369	533920295.00 VO THI KIM TIEN
11012006	101102	10158269574.00 NGUYEN TRAN THANG TRUNG
24052006	080808	325394065.00 PHAN NGUYEN ANH VU
18052006	593672	10000000.00 NGUYEN NGO MAI AN
25161907	294723	10002646.00 TRA QUOC NGUYEN
26281464	242912	21846431.00 NGUYEN VAN A
19472928	232425	2428914.00 NGUYEN DAC CAO DUONG

Hình 1. Một số tài khoản từ file input

Output: Chương trình hiển thị các thông tin liên quan đến quá trình giao dịch trên máy ATM, bao gồm:

- Giao diện menu lựa chọn chức năng giao dịch.
- Kết quả các thao tác kiểm tra số dư, rút tiền, gửi tiền.
- Thông báo lỗi nếu nhập sai thông tin tài khoản, mã PIN, hoặc số tiền không hợp lệ.
- Thông báo khi vượt quá số lần giao dịch hoặc thời gian phiên.
- Các giao dịch được lưu vào file transaction.txt để làm lịch sử giao dịch.

3.2. Cấu trúc dữ liệu

Việc thực hiện ứng dụng xoay quanh một cấu trúc dữ liệu chính là struct CUSTOMER (tài khoản ngân hàng), dùng để lưu trữ tạm thời các thông tin cơ bản như: số tài khoản, mật khẩu, số dư và họ tên của người dùng.

```
struct CUSTOMER {  
    char So_tai_khoan[20];  
    char Mat_khau[15];  
    double So_du;  
    char ten[50];  
};
```

Hình 2. Struct CUSTOMER

Ngoài ra, để quản lý hiệu quả các dữ liệu ấy, chương trình còn sử dụng mảng cấu trúc


```
int cnt = 0;

struct CUSTOMER Tai_khoan[MAX_ACC];
```

Hình 3. Mảng cấu trúc

3.3. Thuật toán

3.3.1. Thuật toán đọc thông tin khách hàng từ file

Input : file dữ liệu khách hàng.

Output : Mảng struct CUSTOMER và số lượng khách hàng.

Bước 1: Mở file, kiểm tra lỗi.

Bước 2: Khởi tạo biến đếm count = 0 để đếm số tài khoản đã đọc

Bước 3: Duyệt từng dòng trong file bằng vòng lặp While

○Lặp lại khi count < Max_acc và fscanf() đọc được đúng 4 trường:

- So_tai_khoan (chuỗi)
- Mat_khau (chuỗi)
- So_du (số thực)
- ten (chuỗi có thể chứa khoảng trắng, đọc đến hết dòng)

○Nếu đọc thành công, tăng count.

Bước 4: Đóng file sau khi đọc xong hoặc gặp lỗi.

Bước 5: Trả về count – số lượng tài khoản đã đọc được.

➤Độ phức tạp thời gian $O(n)$ với n là số tài khoản trong file.

3.3.2. Thuật toán tìm tài khoản theo số tài khoản

Input: mảng struct CUSTOMER, số lượng tài khoản trong mảng và số tài khoản cần tìm.

Output: Tài khoản có trong mảng hay không.

Bước 1: Duyệt qua từng phần tử trong mảng Tai_khoan[] từ $i = 0$ đến $i < \text{count}$.

Bước 2: Tại mỗi bước lặp (vòng lặp for):

- So sánh chuỗi `Tai_khoan[i].So_tai_khoan` với chuỗi `So_tai_khoan` (số tài khoản cần tìm).
- So sánh bằng hàm `strcmp` (nếu hai chuỗi giống nhau, `strcmp` trả về 0).

Bước 3: Nếu tìm thấy khớp:

Trả về chỉ số `i` của phần tử đó (tức là vị trí trong mảng).

Bước 4: Nếu duyệt hết mà không thấy tài khoản phù hợp:

Trả về -1 để báo hiệu không tìm thấy.

➤ Độ phức tạp thời gian $O(n*m)$ với n là số tài khoản, m là độ dài chuỗi.

3.3.3. Thuật toán lưu giao dịch vào một file

Input: Số tài khoản đang thực hiện giao dịch, số tiền giao dịch, loại giao dịch.

Output: Giao dịch được ghi vào file.

Bước 1: Khai báo biến đếm tĩnh `tst` (đếm số lần giao dịch), **`tst = 1`**:

Bước 2: Mở file `transaction.txt`:

- Nếu file không tồn tại, hệ thống sẽ tự tạo file mới.
- Nếu không thể mở file, thông báo lỗi và thoát hàm.

Bước 3: Ghi thông tin giao dịch vào file theo định dạng.

Bước 4: Tăng biến `tst` lên 1 (tăng số lần giao dịch).

Bước 5: Đóng file.

➤ Độ phức tạp thời gian $O(1)$

3.3.4. Thuật toán xác thực tài khoản

Input: mảng chứa danh sách tất cả tài khoản người dùng và số lượng tài khoản, dữ liệu nhập từ bàn phím.

Output: một số nguyên cho biết xác thực thành công hay thất bại. Nếu thành công thì cho biết vị trí của tài khoản trong danh sách.

❖ Xác thực số tài khoản:

Bước 1: Khởi tạo biến `tam = 1` (dùng làm cờ lặp), `gia_tri = -1`;

Bước 2: Tại mỗi vòng lặp `While` (khi `tam == 1`):

- Yêu cầu nhập `So_tai_khoan_2`.

- Tìm vị trí tài khoản bằng `tim_tai_khoan(Tai_khoan, count, So_tai_khoan_2)` và gán kết quả cho `gia_tri`.
- Nếu `gia_tri != -1` (tài khoản tồn tại):
 - Đặt `tam = 0`.
 - Thoát vòng lặp.
- Ngược lại (tài khoản không tồn tại):
 - In thông báo lỗi "Tài khoản không tồn tại, vui lòng nhập lại".
 - Tiếp tục vòng lặp để nhập lại.

❖ **Xác thực mã PIN:**

Bước 3: Đặt biến đếm `gioi_han = 5` (số lần được nhập sai tối đa).

Bước 4: Tại mỗi vòng lặp `While` (khi `gioi_han > 0`)

- Yêu cầu người dùng nhập mã PIN (`Mat_khau_2`) bằng cách gọi hàm `che_mk()`.
- So sánh `Mat_khau_2` với `Tai_khoan[gia_tri].Mat_khau`:
 - Nếu đúng trả về `gia_tri` (xác thực thành công).
 - Nếu sai:
 - Nếu còn lượt (`gioi_han > 1`) hiển thị thông báo sai mã PIN và yêu cầu nhập lại.
 - Nếu không còn lượt hiển thị thông báo “Nhập quá số lần cho phép”.

Bước 5: Nếu sau 5 lần vẫn nhập sai mã PIN, thoát vòng lặp và trả về -1.

➤ Độ phức tạp thời gian $O(k*n)$ với k là số lần nhập sai, n là số tài khoản.

3.3.5. Thuật toán kiểm tra số dư

Input: con trỏ trỏ tới tài khoản khách hàng đang đăng nhập.

Output: hiển thị số dư hiện tại ra màn hình, ghi lịch sử giao dịch vào file.

Bước 1: Lấy số dư hiện tại từ tài khoản được truyền vào.

Bước 2: Hiển thị số dư ra màn hình.

Bước 3: Gọi hàm `luu_giao_dich` để lưu thông tin số tài khoản và số dư vào file giao dịch.

➤ Độ phức tạp thời gian $O(1)$.

3.3.6. Thuật toán thực hiện rút tiền

Input: con trở tới tài khoản khách hàng đang đăng nhập và số tiền cần rút.

Output:

- Trường hợp 1: Rút thành công thì hiển thị “Rut tien thanh cong!” và số dư mới. Ghi lịch sử giao dịch vào file.
- Trường hợp 2: Số dư không đủ thì hiển thị “So du khong du!”.
- Trường hợp 3: Nhập số tiền không dương thì hiển thị “So tien khong hop le!”.

Bước 1: Nhập tmp (số tiền cần rút).

Bước 2: So sánh:

- Nếu $tmp > Tai_khoan \rightarrow So_du$ thì in "Số dư không đủ" và dừng.
- Nếu $tmp \leq 0$ in “Số tiền không hợp lệ”
- Ngược lại:
 - $Tai_khoan \rightarrow So_du = Tai_khoan \rightarrow So_du - tmp$.
 - In "Rút tiền thành công" và số dư mới.
 - Gọi hàm `luu_giao_dich` để lưu thông tin số tài khoản và số dư vào file giao dịch.

Bước 3: Kết thúc.

➤ Độ phức tạp thời gian $O(1)$.

3.3.7. Thuật toán thực hiện gửi tiền

Input: con trở tới tài khoản khách hàng đang đăng nhập và số tiền cần gửi.

Output:

- Trường hợp gửi tiền thành công thì hiển thị “Gui tien thanh cong” và số dư mới. Ghi lịch sử giao dịch vào file.
- Trường hợp nhập sai(số tiền không dương) thì hiển thị “So tien gui khong hop le!”.

Bước 1: Nhập tmp (số tiền cần gửi).

Bước 2: So sánh:

- Nếu $tmp \leq 0$ thì in "Số tiền gửi không hợp lệ" và dừng.
- Ngược lại:
 - $Tai_khoan \rightarrow So_du = Tai_khoan \rightarrow So_du + tmp$.

- In "Gửi tiền thành công" và số dư mới.
- Gọi hàm `luu_giao_dich` để lưu thông tin số tài khoản và số dư vào file giao dịch.

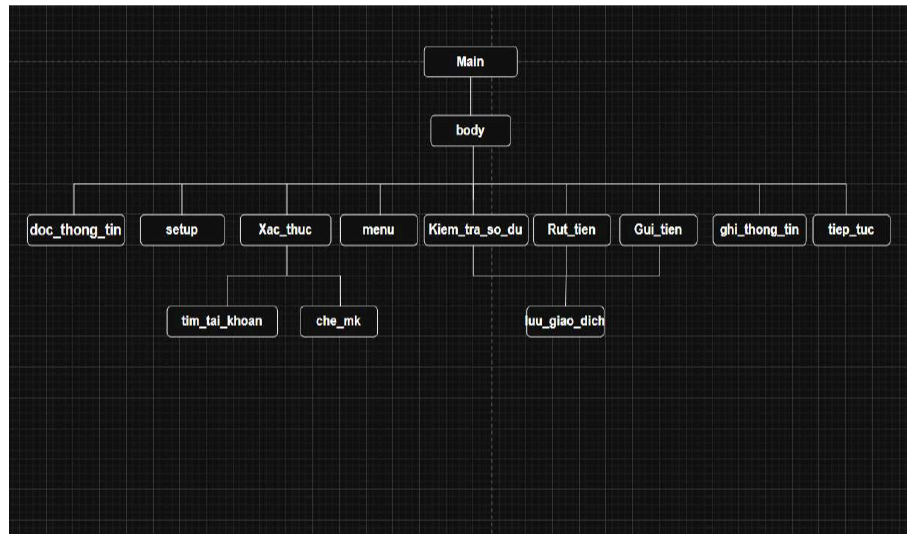
Bước 3: Kết thúc.

➤ Độ phức tạp thời gian $O(1)$.

4. CHƯƠNG TRÌNH VÀ KẾT QUẢ

4.1. Tổ chức chương trình

4.1.1. Sơ đồ tổ chức chương trình



Hình 4. Sơ đồ tổ chức chương trình

4.1.2. Xây dựng chương trình

- Cấu trúc dữ liệu:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
```

```
#define MAX_ACC 100

const int gioi_han_giao_dich = 6; // số lần giới hạn giao dịch

struct CUSTOMER {

    char So_tai_khoan[20];

    char Mat_khau[15];

    double So_du;

    char ten[50];

};
```

- Nguyên mẫu hàm:

```
//
void body();
int doc_thong_tin(struct CUSTOMER[], int);
void setup();
int Xac_thuc(struct CUSTOMER[], int);
void menu();
void Kiem_tra_so_du(struct CUSTOMER*);
void Rut_tien(struct CUSTOMER*);
void Gui_tien(struct CUSTOMER*);
void ghi_thong_tin(struct CUSTOMER[], int);
int tiep_tuc();
int tim_tai_khoan(struct CUSTOMER[], int, char[]);
void che_mk(char*);
void luu_giao_dich(char*, double, const char*);
```

Hình 5. Nguyên mẫu hàm của chương trình

4.2. Ngôn ngữ cài đặt

Đồ án của nhóm được cài đặt bằng ngôn ngữ C.

4.3. Kết quả

4.3.1. Giao diện chính của chương trình



Hình 6. Giao diện giới thiệu đề tài và thành viên



Hình 7. Giao diện chính của chương trình

4.3.2. Kết quả thực thi của chương trình

```
Nhap so tai khoan: 12356
Tai khoan khong ton tai,vui long nhap lai!
Nhap so tai khoan: 12345
Nhap ma PIN: ****
Ma PIN khong dung, vui long nhap lai!
Nhap ma PIN: ***

Xin Chao - VO THI KIM TIEN!
```

Hình 8. Giao diện chức năng đăng nhập và xác thực tài khoản

```
+=====+
|                                     |
|                               MENU CHUC NANG                               |
|                                     |
| 1. Kiem tra so du               3. Gui tien                             |
| 2. Rut tien                     4. Thoat                             |
|                                     |
+=====+

Nhap lua chon: 1

+=====+
|                                     |
| 1. Tiep tuc                     |
| 2. Quay lai                     |
|                                     |
+=====+

Nhap lua chon: 1

So du hien tai: 234071292.00
```

Hình 9. Kết quả chức năng kiểm tra số dư


```
+=====+
|           MENU CHUC NANG           |
+=====+
| 1. Kiem tra so du      3. Gui tien  |
| 2. Rut tien            4. Thoat    |
+=====+

Nhap lua chon: 2

+=====+
| 1. Tiep tục            |
| 2. Quay lai           |
+=====+

Nhap lua chon: 1

Nhap so tien can rut: 24643532
Rut tien thanh cong!
So du moi: 209427760.00 VND
```

Hình 10. Kết quả chức năng rút tiền

```
Xin Chao - VO THI KIM TIEN!
+=====+
|           MENU CHUC NANG           |
+=====+
| 1. Kiem tra so du      3. Gui tien  |
| 2. Rut tien            4. Thoat    |
+=====+

Nhap lua chon: 2

+=====+
| 1. Tiep tục            |
| 2. Quay lai           |
+=====+

Nhap lua chon: 1

Nhap so tien can rut: 32415426424332
So du khong du!
```

Hình 11. Kết quả khi số tiền cần rút nhiều hơn số dư

```
+=====+
|           MENU CHUC NANG           |
+=====+
| 1. Kiem tra so du      3. Gui tien  |
| 2. Rut tien            4. Thoat    |
+=====+

Nhap lua chon: 2

+=====+
| 1. Tiep tục             |
| 2. Quay lại            |
+=====+

Nhap lua chon: 1

Nhap so tien can rut: -111324243
So tien khong hop le!
```

Hình 12. Kết quả khi nhập số tiền cần rút không dương

```
+=====+
|           MENU CHUC NANG           |
+=====+
| 1. Kiem tra so du      3. Gui tien  |
| 2. Rut tien            4. Thoat    |
+=====+

Nhap lua chon: 3

+=====+
| 1. Tiep tục             |
| 2. Quay lại            |
+=====+

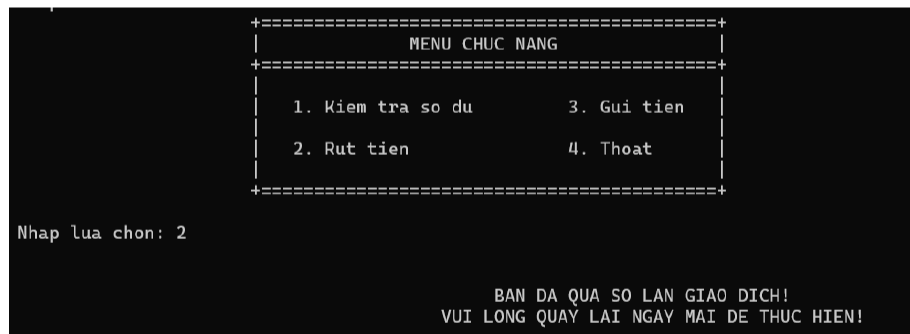
Nhap lua chon: 1

Nhap so tien can gui: 327463243
Gui tien thanh cong
So du moi: 536890992.00 VND
```

Hình 13. Kết quả chức năng gửi tiền



Hình 14. Kết quả khi nhập số tiền cần gửi không dương



Hình 15. Kết quả thực hiện quá số lần giao dịch

```
[Kiem tra so du] So tai khoan: 10022006 - So du: 533945647.000 VND
[Rut tien] So tai khoan: 10022006 - So du: 533920295.000 VND
[Kiem tra so du] So tai khoan: 11012006 - So du: 247837492.000 VND
[Gui tien] So tai khoan: 11012006 - So du: 249980038.000 VND
[Gui tien] So tai khoan: 11012006 - So du: 35504632966.000 VND
[Rut tien] So tai khoan: 11012006 - So du: 10158269574.000 VND
[Kiem tra so du] So tai khoan: 18052006 - So du: 456789.000 VND
[Gui tien] So tai khoan: 18052006 - So du: 325719317.000 VND
[Rut tien] So tai khoan: 18052006 - So du: 325394065.000 VND
[Kiem tra so du] So tai khoan: 19472928 - So du: 2428914.000 VND
[Kiem tra so du] So tai khoan: 19472928 - So du: 2428914.000 VND
[Rut tien] So tai khoan: 19472928 - So du: 1993451.000 VND
[Gui tien] So tai khoan: 19472928 - So du: 45628215.000 VND
```

Hình 16. Số giao dịch được lưu vào file

4.3.3. Nhận xét đánh giá

- Chương trình có đầy đủ các chức năng của một máy ATM.
- Cấu trúc hợp lý, có xử lý các trường hợp lỗi cơ bản.
- Giao diện người dùng đơn giản, dễ sử dụng.
- Tuy nhiên chương trình chưa có chức năng đăng nhập của nhân viên trong ngày.

5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Nhìn chung, dự án đã phát triển thành công các chức năng cơ bản mô phỏng của một máy giao dịch ATM bằng C. Giao diện người dùng dạng văn bản thân thiện, dễ sử dụng, giúp minh họa rõ quy trình vận hành của hệ thống ATM. Hệ thống đảm bảo tính chính xác với kiểm tra dữ liệu và lưu dữ liệu qua tệp. Tuy chương trình mới chỉ đáp ứng các chức năng cơ bản nhưng đây là nền tảng quan trọng để phát triển các tính năng nâng cao như: xác thực 2 bước, giới hạn rút tiền, giao diện đồ họa,..

5.2. Hướng phát triển trong tương lai

Chương trình có thể được phát triển thêm các tính năng nâng cao như:

- Giao diện người dùng đồ họa (GUI) thay vì chỉ dùng giao diện dòng lệnh.
- Sử dụng cơ sở dữ liệu (SQL) thay thế tệp để quản lý dữ liệu lớn.
- Bổ sung các chức năng chuyển khoản giữa các tài khoản với nhau.
- Mô phỏng hoạt động đa người dùng.
- Nâng cấp tính năng bảo mật

TÀI LIỆU THAM KHẢO

- [1] Trần Xuân Lộc, *Nhập và xuất dữ liệu trong C*, <https://blog.28tech.com.vn/c-printf-va-scanf-nhap-xuat-du-lieu>
- [2] Trần Xuân Lộc, *Struct – Kiểu cấu trúc*, <https://blog.28tech.com.vn/c-struct-kieu-cau-truc>
- [3] Trần Xuân Lộc, *Làm việc với file text trong C*, <https://blog.28tech.com.vn/c-doc-ghi-file-text>
- [4] Trần Xuân Lộc, *Mảng cấu trúc*, <https://blog.28tech.com.vn/c-mang-cau-truc>

PHỤ LỤC

```
#include <stdio.h>

#include <conio.h>

#include <stdlib.h>

#include <string.h>

#include <windows.h>

#define MAX_ACC 100

const int gioi_han_giao_dich = 6;

struct CUSTOMER {

    char So_tai_khoan[20];

    char Mat_khau[15];

    double So_du;

    char ten[50];

};

void body();

int doc_thong_tin(struct CUSTOMER[], int);

void setup();

int Xac_thuc(struct CUSTOMER[], int);

void menu();
```



```
        Kiem_tra_so_du(&Tai_khoan[tamp] );

        break;

    }

    else {

        break;

    }

break;

case 2:

printf("\t\t\t\t\t +=====+\n");

printf("\t\t\t\t\t |                               |\n");

printf("\t\t\t\t\t |          1. Tiep tục          |\n");

printf("\t\t\t\t\t |                               |\n");

printf("\t\t\t\t\t |          2. Quay lai          |\n");

printf("\t\t\t\t\t |                               |\n");

printf("\t\t\t\t\t +=====+\n\n");

printf("\t\t\t\t\t Nhap lua chon: ");

int m;

scanf("%d",&m);

printf("\n");

if( m==1) {

        Rut_tien(&Tai_khoan[tamp]);
```

```
        break;

    }

    else {

        break;

    }

    case 3:

        printf("\t\t\t\t\t +=====+\n");

        printf("\t\t\t\t\t |                               |\n");

        printf("\t\t\t\t\t |           1. Tiep tục           |\n");

        printf("\t\t\t\t\t |                               |\n");

        printf("\t\t\t\t\t |           2. Quay lai           |\n");

        printf("\t\t\t\t\t |                               |\n");

        printf("\t\t\t\t\t +=====+\n\n");

        printf("\t\t\t\t\t Nhap lua chon: ");

        int b;

        scanf("%d",&b);

        if( b==1) {

            Gui_tien(&Tai_khoan[tamp]);

            break;

        }

        else {
```

[illegible]

[illegible]

```
=====
=====\\n");

printf("    |                                TRUONG DAI HOC BACH
KHOA DA NANG                                |\\n");

printf("
=====
=====

=====\\n");

printf("    |                                Hoc phan PBL1: Du an lap trinh tinh
toan                                |\\n");

printf("    |                                XAY DUNG UNG DUNG THUC
HIEN GIAO DICH TREN MAY ATM                                |\\n");

printf("    | Nguai huong dan:                                GVC.ThS. Tran Ho
Thuy Tien                                |\\n");

printf("    | Sinh vien thuc hien: Phan Nguyen Anh Vu    MSSV:
102240122    LOP: 24T_DT2                                |\\n");

printf("    |                                Vo Thi Kim Tien    MSSV: 102240113
LOP: 24T_DT2                                |\\n");

printf("    |                                NHOM: 24.11B
|\\n");

printf("
=====
=====

=====\\n");
}
```

```
int Xac_thuc(struct CUSTOMER Tai_khoan[], int count ) {

    int gia_tri=-1,tam=1;

    char So_tai_khoan_2[20], Mat_khau_2[15];

    while(tam) {

        printf("\t\t\t Nhập so tai khoan: ");

        scanf("%s", So_tai_khoan_2);

        gia_tri = tim_tai_khoan(Tai_khoan , count , So_tai_khoan_2);

        if( gia_tri != -1) {

            tam=0;

            break;

        }

        else {

            printf("\n\t\t\t Tài khoản không tồn tại,vui long nhập lại!\n\n");

        }

    }

    int gioi_han = 5;

    while (gioi_han-- ) {

        printf("\n\t\t\t Nhập ma PIN: ");

        che_mk(Mat_khau_2);

        if( strcmp( Tai_khoan[gia_tri].Mat_khau , Mat_khau_2) == 0 ){

            return gia_tri;

        }

    }

}
```



```
printf("\t\t\t\t | \n");  
  
printf("\t\t\t\t  
+=====+\n\n");  
  
printf("\t\t Nhap lua chon: ");  
  
}  
  
void Kiem_tra_so_du(struct CUSTOMER *Tai_khoan) {  
  
    printf("\t\t\t\t So du hien tai: %0.3f\n\n", Tai_khoan->So_du);  
  
    luu_giao_dich(Tai_khoan->So_tai_khoan, Tai_khoan->So_du, "Kiem tra  
so du");  
  
}  
  
void Rut_tien(struct CUSTOMER *Tai_khoan) {  
  
    float tmp;  
  
    printf("\t\t\t\t Nhap so tien can rut: ");  
  
    scanf("%f" , &tmp);  
  
    if( tmp > Tai_khoan->So_du ) {  
  
        Sleep(300);  
  
        printf("\t\t\t\t So du khong du!\n\n");  
  
    }  
  
    else if (tmp <= 0) {  
  
        printf("\t\t\t\t So tien khong hop le!\n");
```

```
        return;

    }

    else {

        Tai_khoan->So_du -= tmp;

        Sleep(300);

        printf("\n\t\t\t\t\t Rut tien thanh cong!\n\n");

        Sleep(200);

        printf("\t\t\t\t\t So du moi: %0.3f VND\n\n",Tai_khoan->So_du );

        luu_giao_dich(Tai_khoan->So_tai_khoan, Tai_khoan->So_du, "Rut
        tien");

    }

}

void Gui_tien(struct CUSTOMER *Tai_khoan) {

    float tmp;

    printf("\n\t\t\t\t\t Nhap so tien can gui: ");

    scanf("%f" , &tmp);

    if(tmp <= 0 ) {

        Sleep(300);

        printf("\n\t\t\t\t\t So tien gui khong hop le!\n\n");

    }

}
```

```
else {  
    Tai_khoan->So_du += tmp;  
    Sleep(300);  
    printf("\n\t\t\t\t\t Gui tien thanh cong\n");  
    Sleep(300);  
    printf("\n\t\t\t\t\t So du moi: %0.2f VND\n\n", Tai_khoan->So_du );  
    luu_giao_dich(Tai_khoan->So_tai_khoan,Tai_khoan->So_du, "Gui  
tien");  
}  
}  
  
void ghi_thong_tin(struct CUSTOMER Tai_khoan[], int count) {  
    FILE *f=fopen("accounts.txt","w");  
    if (f == NULL) {  
        printf("\t\t\t\t\t Loi mo file\n");  
        return;  
    }  
    int i;  
    for (i=0;i<count;i++) {  
        fprintf(f,"%s %s %0.2f %s\n",  
            Tai_khoan[i].So_tai_khoan ,
```



```
int i=0;

char c;

while(1) {

    c = getch();

    if( c == 13 ) {

        PASS[i] = '\0';

        break;

    }

    else if( c == 8 && i > 0 ) {

        i--;

        printf("\b \b");

    }

    else if( i < 14 && c >= 32 && c <=126 ) {

        PASS[i++] = c;

        printf("*");

    }

}

printf("\n");

}
```

